

Application Development

Lecture 3: Object-Oriented Applications

Konstantin Kapinchev
University of Greenwich

Application Development

Lecture 3: Object-Oriented Applications

In this lecture:

- **Motivations**
- **Classes and Objects**
- **Data and Functionality**
- **Access Modifiers**
- **Constructors**
- **Static Classes**
- **Example**
- **Conclusions**

Application Development

Lecture 3: Object-Oriented Applications

Motivations

- **Object-Oriented Programming allows us to:**
 - **Organise larger programs into smaller self-contained units**
 - **Manage efficiently these units**
 - **Incorporate both data and functionality into the units**
 - **Re-use these units in various circumstances**
 - **Improve code security**

Application Development

Lecture 3: Object-Oriented Applications

Classes and Objects

- **The concepts of classes and objects underpin the object-oriented programming**
- **A class is a user-defined type¹**
- **It provides a description of:**
 - **Data (variables)**
 - **Functionality (methods)**

¹As opposed to built-in data types

Application Development

Lecture 3: Object-Oriented Applications

Classes and Objects *continued*

- An object is an instance of a class¹
- In most cases, instantiation is necessary before utilising functionality from classes²
- Multiple objects can be created by using the same class
- Compared with classes, objects require more resources³

¹Some textbooks compare classes to blueprints of buildings and objects to the actual buildings

²Except static classes

³Most notably, memory

Application Development

Lecture 3: Object-Oriented Applications

Data and Functionality

- **Classes consist of data and functionality**
 - **Data¹, in the form of variables**
 - **Functionality¹, in the form of methods**

¹Some sources call data attributes and functionality behaviour

```

using System;

namespace Project
{
    class MyClass
    {
        // Data
        private double? Z = null;

        // Functionality
        public void Logarithm(double X, double Y)
        {
            Z = Math.Log(X, Y);
        }
        public double? GetResult()
        {
            return (Z);
        }
    }

    public class MyProject
    {
        static void Main(string[] args)
        {
            double A = 8;
            double B = 2;

            MyClass MyObject = new MyClass();

            MyObject.Logarithm(A, B);

            Console.WriteLine("The logarithm of {0} to the base {1} is {2}", A, B, MyObject.GetResult());
        }
    }
}

```

Example:

Class MyClass contains data, *variable Z*, which has the result of the calculations, and the functionality, *method Logarithm*, which performs the calculations. Additional method, *GetResult*, provides access to the data.

The type of Z is "double?", which allows the variable to store real numbers and a value called "null". This is useful when calculations might not produce a result, for example when calculating logarithm of a negative value.

Application Development

Lecture 3: Object-Oriented Applications

Access Modifiers

- Access to the data and the functionality of a class from the rest of the program is determined by keywords called Access Modifiers
- The following are the access modifiers in C#:
 - **public**¹
 - **private**¹
 - **protected**¹
 - **internal**
 - **protected internal**
 - **private protected**

¹Those are the more common access modifiers

Application Development

Lecture 3: Object-Oriented Applications

Access Modifiers *continued*

Location of the caller	public	protected internal	protected	internal	private protected	private
Within the class	✓	✓	✓	✓	✓	✓
Derived class, same assembly	✓	✓	✓	✓	✓	×
Non-derived class, same assembly	✓	✓	×	✓	×	×
Derived class, different assembly	✓	✓	✓	×	×	×
Non-derived class, different assembly	✓	×	×	×	×	×

Application Development

Lecture 3: Object-Oriented Applications

Constructors

- **Type of methods, which are automatically called when an object is created**
- **Constructors have the same name as their corresponding classes**
- **Constructors do not return values**
- **Constructors can have parameters**
- **They are usually developed for the initialisation of the class variables**

Application Development

Lecture 3: Object-Oriented Applications

Constructors *continued*

- **Classes can have multiple constructors**
- **One of the constructors is the "default constructor"¹**
- **In case of a class with multiple constructors, which constructor to run is determined based on the provided parameter list, as illustrated in the example on the following slides**

¹The term "default constructor" has different meanings in different programming languages, in C# this is a constructor without parameters

Application Development

Lecture 3: Object-Oriented Applications

Constructors *continued*

- Example
 - Define 3 constructors with different parameters:

```
public class MyClass
{
    public MyClass()
    {
        Console.WriteLine("Default Constructor");
    }

    public MyClass(int i, int j)
    {
        Console.WriteLine("Constructor Calculating {0} + {1} = {2}", i, j, i + j);
    }

    public MyClass(int i, double j)
    {
        Console.WriteLine("Constructor Calculating {0} * {1} = {2}", i, j, i * j);
    }
}
```

Application Development

Lecture 3: Object-Oriented Applications

Constructors *continued*

- Example

- Create 3 objects:

```
MyClass MyObject1 = new MyClass();  
MyClass MyObject2 = new MyClass(2, 3);  
MyClass MyObject3 = new MyClass(2, 3.0);
```

- Output generated by the 3 constructors:

```
Default Constructor  
Constructor Calculating 2 + 3 = 5  
Constructor Calculating 2 * 3 = 6
```

Note: For "MyObject2" and "MyObject3" the data type alone determines which constructor to run

Application Development

Lecture 3: Object-Oriented Applications

Static Classes

- Type of class which cannot be instantiated¹
- Declared by using the "static" keyword
- In a static class all data and methods need to be static as well²

¹There cannot be objects based on static classes

²The following error will be generated if non static data or methods are declared in a static class: "cannot declare instance member in a static class"

Application Development

Lecture 3: Object-Oriented Applications

Static Classes *continued*

- When to use static classes:
 - For tasks which do not require objects
 - For functionality which generates deterministic results¹
 - Suitable for mathematical tasks

¹Given a particular input, always produce the same output

Application Development

Lecture 3: Object-Oriented Applications

Static Classes *continued*

- Non static classes can contain static data and methods
- Those data and methods cannot be accessed via objects¹
- Although object instantiation is allowed
- As a result, static variables have the same values throughout all objects, which are instances of that class

¹This would generate the error: "Cannot be accessed with an instance reference, qualify it with a type name instead"

Application Development

Lecture 3: Object-Oriented Applications

Static Classes *continued*

- Examples:
 - The "Main" method, which is the entry point of a C# program
 - It does not have an object

```
static void Main(string[] args)
```

Application Development

Lecture 3: Object-Oriented Applications

Static Classes *continued*

- Examples:
 - Functionality which is expected to work regardless of the objects' states, commonly utilised for mathematical tasks

```
public static double CalculateSquare(double x)
{
    return (x * x);
}
```

Application Development

Lecture 3: Object-Oriented Applications

Example

- An object-oriented implementation of quadratic equation
- Class QuadraticEquation provides the solutions
- The class has:
 - 5 private variables
 - A default constructor
 - 3 public methods
 - Method "Calculate" finds the solutions based on its parameters
 - Methods "GetX1" and "GetX2" provide access to solutions x1 and x2

Application Development

Lecture 3: Object-Oriented Applications

Example *continued*

- An object-oriented implementation of quadratic equation

```
public class QuadraticEquation
{
    // Private variables
    private double a;
    private double b;
    private double c;
    private double? x1;
    private double? x2;

    //Default constructor
    public QuadraticEquation()
    {
        a = Convert.ToDouble(0);
        b = Convert.ToDouble(0);
        c = Convert.ToDouble(0);
        x1 = null;
        x2 = null;
    }
}
```

Application Development

Lecture 3: Object-Oriented Applications

Example *continued*

- An object-oriented implementation of quadratic equation

```
// Public Methods
// Calculate the solutions to the equation
public void Calculate(double a, double b, double c)
{
    double D;
    D = b * b - 4 * a * c;
    if (D >= 0)
    {
        x1 = Convert.ToDouble((-b + Math.Sqrt(D)) / (2 * a));
        x2 = Convert.ToDouble((-b - Math.Sqrt(D)) / (2 * a));
    }
    else
    {
        x1 = null;
        x2 = null;
    }
}
```

Application Development

Lecture 3: Object-Oriented Applications

Example *continued*

- An object-oriented implementation of quadratic equation

```
// Access the value of x1
public double? GetX1()
{
    return (x1);
}

// Access the value of x2
public double? GetX2()
{
    return (x2);
}

} //End of class
```

Application Development

Lecture 3: Object-Oriented Applications

Example *continued*

- An object-oriented implementation of quadratic equation

```
static void Main(string[] args)
{
    // Create an object "MyObject" from the class "QuadraticEquation"
    QuadraticEquation MyObject = new QuadraticEquation();
    // Provide coefficients a, b and c
    // Calculate x1 and x2
    MyObject.Calculate(1, -5, 6);
    // Display the values of x1 and x2 on the screen
    Console.WriteLine("The results are: {0}, {1}", MyObject.GetX1(), MyObject.GetX2());
}
```

Application Development

Lecture 3: Object-Oriented Applications

Conclusions

- **This lecture introduced the main concepts of the object-oriented programming**
- **C# examples illustrated these concepts**
- **Most of these concepts will be discussed in more details when they are employed in advanced object-oriented techniques**
- **The benefits of the OOP were discussed and demonstrated**